Perfect. This is where we stop thinking like product designers and start thinking like software engineers.

This document is what I'd personally rely on before writing a single line of code.

---

Assignment Guy

Document 4

Technical Design Document (TDD)

Version 1.0

---

1. Purpose

This document describes how Assignment Guy should be built from a software engineering perspective.

It defines the architecture, backend responsibilities, authentication, storage, scalability, security, and major technical decisions.

The goal is to ensure the application remains maintainable as it grows.

---

2. Technical Philosophy

Version 1 should prioritize:

Simplicity

Reliability

Maintainability

Security

Scalability

The first version should avoid unnecessary complexity while remaining easy to expand.

---

3. Platform Support

Version 1

✅ Android

Future

iOS

Web

Desktop

Mobile is the primary experience.

---

4. Suggested Technology Stack

Frontend

React + TypeScript

Responsive mobile-first design

---

Backend

Node.js

Express.js

REST API

---

Database

PostgreSQL

Reason:

Reliable

Relational

Excellent querying

Future-proof

---

Authentication

Supabase Auth

or

Firebase Authentication

---

File Storage

Cloud object storage.

Stores:

Images

PDFs

Future voice notes

---

Push Notifications

Firebase Cloud Messaging (FCM)

---

5. Architecture

Client

↓

REST API

↓

Authentication

↓

Database

↓

Storage

↓

Notifications

Every responsibility should remain separated.

---

6. Backend Responsibilities

The backend handles:

Authentication

Course management

Assignments

Uploads

Comments

Notifications

Contribution tracking

Status updates

Reports

Moderation

Never trust client-side validation alone.

---

7. Frontend Responsibilities

The frontend handles:

Displaying information

Input validation

Navigation

Searching

Uploading files

Caching

User interactions

Business rules should remain on the server whenever possible.

---

8. Authentication

Users must authenticate before accessing private features.

Supported methods:

Email

Google

School email (future)

Authentication should remain independent of user profile information.

---

9. User Profile

Each user has:

User ID

Display Name

Profile Picture (optional)

School

Department

Level

Joined Courses

Contribution Score

Role

Created Date

---

10. Course Structure

Courses are independent.

Example

Computer Engineering

↓

CPE201

↓

Assignments

↓

Discussion

Students only see courses they join.

---

11. Assignment Structure

Each assignment contains:

Assignment ID

Course

Title

Question

Attachments

Context

Due Date

Uploader

Created Time

Updated Time

Status

---

12. Discussion Structure

Each assignment owns its discussion.

Discussion

↓

Comments

↓

Replies

Simple nesting only.

Avoid deeply nested threads.

---

13. Notification System

Notifications should be event-driven.

Examples:

Assignment Uploaded

↓

Notify enrolled students

Comment Added

↓

Notify participants

Deadline Changed

↓

Notify affected students

Keep notifications lightweight.

---

14. Search

Search should support:

Assignment Title

Course

Topic

Keywords

Future:

Full-text search.

---

15. File Uploads

Version 1 supports:

Images

PDF

Maximum file size should be configurable.

Rejected uploads should display clear errors.

---

16. Caching

Frequently viewed data should be cached.

Examples:

Assignment Feed

Course List

Recent Assignments

This improves performance.

---

17. Offline Behaviour

Version 1:

Limited support.

Recently viewed assignments remain accessible.

Uploading requires internet.

Future:

Offline-first.

---

18. Error Handling

Errors should be categorized.

Client Errors

Validation

Missing fields

Unauthorized

Server Errors

Database

Storage

Unexpected failures

Every error should be logged.

---

19. Logging

Important events:

Login

Assignment Upload

Assignment Edit

Deletion

Reports

Errors

Emergency Answer Usage

Logs help debugging and moderation.

---

20. Security

Never trust client input.

Validate everything.

Sanitize user text.

Prevent SQL injection.

Protect file uploads.

Limit request abuse.

Use HTTPS.

Passwords should never be stored directly.

---

21. Privacy

Students control:

Profile visibility

Assignment status visibility (if you keep it optional)

Notification preferences

Only necessary data should be collected.

---

22. Scalability

Architecture should support future additions without major redesign.

Future examples:

Voice uploads

OCR

Lecturer accounts

Timetable

Calendar

Study groups

Analytics

Because the system is modular, these can be added later.

---

23. AI Integration

Assignment Guy does not perform AI inference.

Instead it:

Collects assignment information

Formats context

Launches the selected external AI service

No AI API dependency exists in Version 1.

This significantly reduces operational costs.

---

24. Emergency Answer

The backend controls all unlocking rules.

The client must never decide whether a user has access.

Possible checks:

Contribution eligibility

Usage limits

Time window

Future conditions

This prevents manipulation.

---

25. Future APIs

The backend should be designed to support:

Public API (future)

Institution integrations

Third-party AI integrations

Calendar sync

without major restructuring.

---

26. Version 1 Technical Goals

The application should comfortably support:

Thousands of users

Hundreds of courses

Thousands of assignments

Without architectural redesign.

Performance optimization beyond this can happen later.

---

End of Document 4

---

My thoughts after writing this

This is a solid foundation, but I deliberately kept it technology-agnostic in places. For example, I mentioned React, Express, PostgreSQL, and Supabase/Firebase as reasonable choices, but I didn't lock the project into them.

The reason is that your implementation plan has two phases:

Version 1: Replit Agent may choose a slightly different stack based on its strengths.

Version 2: You'll likely rebuild parts of it yourself.

Keeping this document focused on architecture and responsibilities rather than specific frameworks makes it useful regardless of whether the implementation evolves. Once we get to the database and API documents, we'll become much more concrete about tables, endpoints, relationships, and data flow. Those documents will feel much closer to an engineering blueprint than a product description.